

第十六章：调试实战——从 Page Fault 到 Root Cause

本章选取项目开发中最有代表性的调试案例，展示从现象到根因的完整排查过程。每个案例都体现了系统级调试的方法论：观察现象 → 缩小范围 → 构建假设 → 验证修复。

案例 1：内核栈溢出 — Store Page Fault

现象

运行基础测试时随机触发 panic：

```
store page fault: scause=0xf, stval=0x3ffffff8000
```

排查过程

第一步：分析 stval。0x3ffffff8000 是一个高地址，落在某个进程的 KSTACK guard page 范围内。Guard page 是故意不映射的，用来检测栈溢出。

第二步：检查 kernel.asm。反汇编发现 fault 时的 sepc 偏移指向 initcode 数据区——说明返回地址已被破坏（栈溢出覆盖了 ra）。

第三步：追溯调用链。FAT32 路径解析 (fat32_namei) → 簇链遍历 → readi → bread → 嵌套了 6-7 层函数，每层几十到几百字节的局部变量。4KB 内核栈不够。

修复

memlayout.h 引入 KSTACK_PAGES=2（8KB），修改 proc_mapstacks 映射 2 页栈 + 1 页 guard。

深度思考

- Linux 默认内核栈 8KB（THREAD_SIZE），CONFIG_KASAN 开启时需要 32KB
- 内核栈溢出是**无声的致命错误**——可能覆盖相邻数据结构而不立即 panic
- Guard page 是栈溢出检测的基本手段，但只在栈增长超过一页时有效（如果栈增长恰好跳过 guard page 直接到下一个有效映射，就检测不到）

案例 2：BusyBox exec 失败 — 用户栈布局

现象

busybox echo "Hello" 运行后：

1. 早期：直接 store page fault（SIGSEGV）
2. 修复部分 syscall 后：打印 "applet not found" 然后退出

排查过程

第一步：加 syscall trace。只对进程名为 "busybox" 的进程打印系统调用号、参数、返回值。发现 exec 后的第一批调用包括 `set_tid_address`、`getuid`、`fstatat` 等未实现的 syscall。

第二步：补齐基础 syscall stub。`set_tid_address` 返回 pid、`getuid` 返回 0、`exit_group` 复用 `exit`、`fstatat` 复用现有 `namei + stat_to_kstat`。

第三步：分析 "applet not found"。BusyBox 通过 `argv[0]` 确定要执行的命令。如果 argv 解析错误，它认为找不到对应的 applet。

第四步：检查栈布局。`exec.c` 构造的初始用户栈不符合 Linux/RISC-V 约定：

```
正确布局: sp → [argc][argv[0]][argv[1]]... [NULL][envp[0]][NULL][auxv...]  
错误布局: argc 和 argv 之间有对齐填充 → libc 读错偏移
```

修复

重新排布 `exec.c` 的栈尾：`argc` 紧贴 `argv` 指针数组，追加空 `envp` 和 `AT_NULL auxv`，整体 16 字节对齐。

深度思考

- RISC-V 用户栈的 `argc/argv/envp/auxv` 布局是 ABI 规范的一部分，任何偏差都会让 libc 的 CRT 代码读错数据
- 补充第三方二进制需要的 syscall stub 是一个迭代过程——先让它启动，再逐步修复功能
- **syscall trace 是调试第三程序最有效的工具**——无需阅读目标程序源码，只需观察它的系统调用行为

案例 3：FAT32 目录满 — 目录链未扩展

现象

全量测试运行到一半，`test_mkdir`、`test_unlink`、`test_mmap` 连续失败：

```
fat32_createat: no free directory entry  
argfd: fd -13 invalid
```

排查过程

第一步：分析日志链。`no free directory entry` → `open` 返回 `-ENOSPC`（负值）→ 上层测试把负值当 fd 使用 → `argfd` 报错 → `mmap` 因无效 fd 触发 `usertrap`。

第二步：理解 FAT32 目录结构。FAT32 目录本质是一个文件，由簇链组成。每个簇内存放固定数量的目录项（32 字节/项）。如果当前簇链内的目录项全部被占用，需要分配新簇并接入链尾。

第三步：检查代码。原有逻辑只在当前目录簇链内查找空闲项，到达链尾就直接返回失败，没有扩展目录链。

修复

新增 `fat_set_clus()` 和 `fat_zero_cluster()`，在目录满时自动分配新簇并更新 FAT 表。

深度思考

- FAT32 的目录是**可变长的文件**，需要像普通文件一样支持动态扩展
- 错误码的传播必须正确——内核返回负 `errno`，用户态必须检查而不是直接当 `fd` 使用
- 文件系统的边界情况测试（目录满、磁盘满、长文件名）往往暴露最深的 bug

案例 4：ext4 读取 ELF 头失败 — 虚拟块设备容量

现象

`ext4_mode=1` 时，`exec brk` 和 `exec chdir` 失败：

```
ext4_readi: fread '/musl/basic/chdir' off 0 len 64 ret 22 rcnt 0
exec: read elf header failed for chdir
```

但 `sleep` 测试正常，FAT32 模式下所有测试正常。

排查过程

第一步：排除镜像问题。`debugfs -R 'stat /musl/basic/brk'` 确认文件存在，`inode=17`，`extents` 起始块号 ~699K。

第二步：追踪 `EINVAL` 来源。`lwext4` 的 `ext4_block_readbytes` 有边界检查：`off + len > bdev->part_size` 则返回 `EINVAL`。

第三步：计算。块号 `699K × 4KB ≈ 2.8GB`。但 `ext4fs_init` 中虚拟块设备的 `part_size` 只设了 1GB → 高块号访问被人为拦截。

第四步：解释 `sleep` 正常。`sleep` 测试程序的 `inode` 恰好在低块号区域，未触发边界检查。

修复

将 ext4 虚拟块设备的逻辑容量从 1GB 放宽至 8GB。

深度思考

- 嵌入第三方库时，**mock/适配层的参数必须覆盖实际使用范围**——逻辑容量不足是一个“无声”的限制
- 只有部分测试失败时，考虑差异因素（文件大小、块号位置、路径深度等），有助于缩小排查范围
- 在文件系统的早期加载路径（`exec` 读 ELF 头）加日志，能快速定位“读不到第一段”的原因

案例 5：动态链接适配 — `PT_INTERP` 与 `auxv`

现象

`musl` 编译的动态链接测试程序 `entry-dynamic.exe` 无法启动。

排查过程

第一步：分析 ELF。动态链接 ELF 有 `PT_INTERP` segment，指向 `/lib/ld-musl-riscv64-sf.so.1`。

第二步：FAT32 限制。FAT32 不支持符号链接，该路径不存在。需要在内核中做路径回退（fallback 到 `/libc.so`）。

第三步：auxv 补齐。musl 的动态链接器需要 `AT_BASE`（解释器加载基址）、`AT_PHDR`（程序 Program Header 地址）、`AT_ENTRY`（程序入口点）等辅助向量。exec 必须在用户栈上构造完整 auxv。

第四步：mprotect。musl 需要 `mprotect` 来实现 RELRO（Relocation Read-Only）段保护。实现 `sys_mprotect` 按页更新 PTE 权限。

修复

1. exec 中解析 PT_INTERP，加载解释器 ELF
2. 构造完整 auxv（AT_BASE/AT_PHDR/AT_PHNUM/AT_PHENT/AT_PAGESZ/AT_ENTRY）
3. 解释器路径 fallback（适配 FAT32 镜像）
4. 实现 sys_mprotect

深度思考

- 动态链接是用户态最复杂的启动流程——涉及 ELF 解析、地址空间布局、符号解析、重定位
- auxv 是内核和用户态 CRT/ld.so 之间的隐式协议，少任何一个字段都可能导致 ld.so 自举失败
- 这个案例涉及内核（exec/mprotect）、文件系统（路径回退）、用户态（ELF/auxv）三层协作

案例 6：sleep 睡死 — nanosleep 唤醒失败

现象

调用 `sleep()` 后进程永远不醒。

排查过程

`sys_nanosleep` 将用户传入的秒/纳秒转换为 ticks，然后 `sleep(channel, lock)` 等待 ticks 到期。问题在于唤醒条件不正确——计时逻辑没有正确处理 tick 溢出，或者定时器中断频率与 tick 假设不匹配。

修复

精确转换纳秒到 ticks：`total_ns = sec * 1e9 + nsec * 1e3`，`n_ticks = ceil(total_ns / 1ms)`。确保定时器中断频率与 TICK_NS 假设一致。

深度思考

- 时间相关的 bug 往往与**假设的时钟频率不一致**有关
- sleep/wakeup 的正确性依赖于定时器中断的可靠触发

调试方法论总结

工具箱

工具	用途
scause / stval / sepc	定位异常类型、触发地址、异常指令
kernel.asm	反汇编，从 sepc 找到对应的 C 代码
kernel.sym	符号表，地址→函数名映射
GDB (gdb-multiarch)	断点、单步、查看内存/寄存器
syscall trace	打印系统调用号/参数/返回值，调试用户态程序
printf 日志	最原始但最灵活的调试手段

排查流程

1. 观察现象：panic 信息、scause、stval、sepc
2. 分类异常：
 - store/load page fault → 内存访问问题（栈溢出/COW/VMA/非法访问）
 - instruction page fault → 代码段映射问题（exec/mmap）
 - illegal instruction → 未实现指令/特权级错误
 - syscall 返回错误值 → 追踪具体 syscall 实现
3. 缩小范围：
 - 哪个进程？（myproc()->name）
 - 哪个地址？（stval → memlayout 对照）
 - 什么操作？（sepc → kernel.asm 反查）
4. 构建假设 → 加日志/GDB 验证 → 修复 → 回归测试

经验法则

1. **先让它能说话**：优先确保 console/printf 可用，这是所有其他调试的基础
2. **缩小差异**：只有部分测试失败时，对比通过/失败用例的差异（文件大小、路径深度、syscall 序列）
3. **syscall trace 是万金油**：调试第三方用户态程序时，观察系统调用序列比读源码更高效
4. **地址空间是线索**：stval 落在什么范围？用户栈？内核栈 guard？mmap 区域？页表页？
5. **错误码传播链**：一个底层的 EINVAL 可能在上层被误解为有效值（如负 errno 当 fd 用），追踪完整传播链